

StormFly Ascent: Detailed Development Roadmap

This document provides a granular, week-by-week breakdown of the development plan for "StormFly Ascent," expanding on the 6-month timeline from the project proposal. Each phase is designed to build upon the last, ensuring a structured and manageable workflow from prototype to final submission.

Phase 1: Foundation & Prototyping (Month 1)

Goal: Establish the project's technical foundation and build a working prototype of the core player movement.

- **Week 1: Project Setup & Pre-Production**
 - [] **Version Control:** Initialize Git repository and push to GitHub. Establish branching strategy (e.g., `main`, `develop`, feature branches).
 - [] **Project Structure:** Create a clean folder structure within Unity (e.g., `_Scripts`, `_Prefabs`, `_Scenes`, `_Art`, `_Audio`).
 - [] **Game Design Document (GDD):** Finalize the GDD, detailing every mechanic, system, and UI flow. This will be the project's source of truth.
 - [] **Initial Scene Setup:** Create a basic "test-bed" scene for prototyping mechanics.
- **Week 2: Player Controller (Input & Physics)**
 - [] **Player Prefab:** Create a basic hoverboard 3D model (or placeholder primitive) and create a Player prefab.
 - [] **Physics Setup:** Add a `Rigidbody` component to the player prefab. Tweak mass, drag, and gravity settings.
 - [] **Input Manager:** Create an `InputManager.cs` script.
 - [] **Controller Script:** Create a `PlayerController.cs` script.
 - [] **Tilt Controls:** Implement code to read accelerometer data (`Input.acceleration`) for left/right movement. Apply forces to the `Rigidbody` based on this input.
 - [] **Device Testing:** Build and deploy to an Android device to test and calibrate tilt sensitivity.
- **Week 3: Player Controller (Core Mechanics)**
 - [] **Vertical Movement:** Implement constant upward force to make the player ascend.
 - [] **Boost Mechanic:** Implement tap-to-boost functionality. On screen tap, apply a temporary, strong upward force (`ForceMode.Impulse`). Add a cooldown or energy mechanic for the boost.
 - [] **Movement Clamping:** Constrain the player's movement to the horizontal boundaries of the play area.
 - [] **Script Architecture:** Refactor the `PlayerController.cs` to be clean and organized, separating movement, input, and boost logic.
- **Week 4: Camera & Scene Prototyping**
 - [] **Camera Controller:** Create a `CameraController.cs` script.

- [] **Camera Follow:** Implement logic for the camera to smoothly follow the player's vertical ascent. Ensure it does not follow horizontal movement to avoid motion sickness.
- [] **Basic Environment:** Create simple placeholder prefabs for hazards (cubes) and collectibles (spheres) to test collisions.
- [] **Prototype Review:** Review the prototype's "feel." Does the movement feel responsive? Is the boost satisfying? Adjust physics and controller values based on feedback.

Phase 2: Core Gameplay Loop (Month 2)

Goal: Build a "Vertical Slice" of the game with one complete gameplay loop.

- **Week 5: Procedural Generation (Manager v1)**
 - [] **PCG Manager:** Create a `LevelManager.cs` or `PCGManager.cs`.
 - [] **Modular Prefabs:** Design and create a few simple, modular "chunk" prefabs that can be spawned and connected vertically.
 - [] **Spawning Logic:** Implement logic to spawn new level chunks ahead of the player as they ascend and destroy chunks that are far below.
 - [] **Endless Generation:** Ensure the spawning creates a seamless, "endless" vertical level.
- **Week 6: Hazards & Collectibles**
 - [] **Hazard Prefabs:** Create prefabs for the first set of hazards (e.g., static floating debris, a simple moving obstacle).
 - [] **Collectible Prefabs:** Create a prefab for the primary "energy shard" collectible.
 - [] **Spawning in Chunks:** Update the `PCGManager` to randomly place hazards and collectibles within the level chunks as they are spawned.
 - [] **Collision Logic:** In `PlayerController.cs`, implement `OnCollisionEnter` or `OnTriggerEnter` to detect collisions with hazards and collectibles.
- **Week 7: Game Loop Manager & Basic UI**
 - [] **Game Manager:** Create a central `GameManager.cs` to manage game states (e.g., `MainMenu`, `Playing`, `Paused`, `GameOver`).
 - [] **Player Health/State:** Implement logic for the player to "die" upon hitting a hazard.
 - [] **Scoring System:** Implement a scoring system. Increment the score based on altitude and energy shards collected.
 - [] **Basic HUD:** Using Unity's UI Canvas, create a simple Heads-Up Display (HUD) to show the current score.
 - [] **Game Over Logic:** When the player dies, transition to a `GameOver` state and display a simple "Game Over" UI panel with the final score.
- **Week 8: Vertical Slice & Review**
 - [] **Integrate All Systems:** Ensure the Player Controller, PCG Manager, and Game Manager all work together flawlessly.
 - [] **Build & Test:** Create a stable Android build of the vertical slice.

- [] **Core Loop Validation:** Playtest extensively. Is the core loop (Ascend, Boost, Collect, Survive) fun and engaging?
- [] **Gather Feedback:** Get feedback from peers on the vertical slice and create a backlog of tasks for improvements.

Phase 3: Content & Scalability (Month 3)

Goal: Expand the content library and make the core systems more robust and scalable.

- **Week 9: Advanced Procedural Generation**
 - [] **Zoning System:** Implement logic in the `PCGManager` to create distinct zones (e.g., Debris Field, Lightning Core). Each zone should have its own set of prefabs and hazards.
 - [] **Difficulty Scaling:** Implement a system to increase difficulty over time (e.g., increase player speed, spawn more hazards, introduce faster obstacles).
 - [] **Prefab Variety:** Create more complex and visually interesting level chunk prefabs.
- **Week 10: Power-Up System**
 - [] **Power-Up Manager:** Create a `PowerUpManager.cs` to handle the logic for activating and deactivating power-ups.
 - [] **Power-Up Types:** Implement the first power-ups (e.g., Shield, Score Multiplier). Create scripts for each power-up's behavior.
 - [] **Power-Up Spawning:** Integrate power-up collectibles into the `PCGManager`'s spawning logic.
 - [] **UI Feedback:** Add UI elements to the HUD to show when a power-up is active and for how long.
- **Week 11: Art Asset Integration (1st Pass)**
 - [] **Model Integration:** Replace placeholder art for the player, hazards, and collectibles with initial 3D models.
 - [] **Texturing & Materials:** Apply basic materials and textures to the new models.
 - [] **Environment Theming:** Start creating distinct visual themes for each procedural zone.
 - [] **Visual Effects (VFX):** Add simple particle effects for boost trails, collecting shards, and collisions.
- **Week 12: Content Library Expansion**
 - [] **New Hazards:** Design and implement new, more complex hazards (e.g., lightning strikes that telegraph their position, turbulent wind zones that push the player).
 - [] **New Power-Ups:** If time permits, add another power-up like a "Shard Magnet."
 - [] **Refine Spawning Rules:** Tweak the procedural generation algorithms to create more interesting and challenging combinations of hazards and collectibles.

Phase 4: Systems & UI Integration (Month 4)

Goal: Build the meta-game systems and create a polished user interface.

- **Week 13: UI/UX - Main Menu & Screens**
 - [] **UI Manager:** Create a `UIManager.cs` to manage all UI panels and transitions.
 - [] **Main Menu:** Design and implement the main menu screen (Play, Store, Settings buttons).
 - [] **Screen Flow:** Implement the navigation logic between the main menu, the game, and the game over screen.
 - [] **Settings Panel:** Create a basic settings panel (e.g., for adjusting music/SFX volume).
- **Week 14: UI/UX - HUD & In-Game Feedback**
 - [] **Polished HUD:** Redesign the in-game HUD to be visually appealing and informative.
 - [] **Pause Menu:** Implement a pause menu with "Resume," "Restart," and "Quit to Menu" options.
 - [] **Animated Feedback:** Add subtle UI animations (e.g., score counter increasing, power-up icon flashing) to make the experience feel more dynamic.
- **Week 15: Data Persistence & Currency**
 - [] **Currency System:** Integrate an in-game currency that is awarded based on performance.
 - [] **Data Manager:** Create a `DataManager.cs` or `SaveLoadManager.cs`.
 - [] **Serialization:** Implement the logic to save and load player data (e.g., total currency, unlocked cosmetics, high score) to a JSON file on the device.
 - [] **Data Hooks:** Ensure the game saves data after a run and loads it when the game starts.
- **Week 16: Checkpoint System**
 - [] **Checkpoint Logic:** Design the logic for how checkpoints work. When does the player reach one? How is progress saved?
 - [] **Checkpoint Prefab:** Create a visual prefab for checkpoints that can be spawned by the `PCGManager`.
 - [] **Game Manager Integration:** Update the `GameManager` to handle respawning the player at the last checkpoint instead of restarting the run.
 - [] **UI for Checkpoints:** Add UI feedback for reaching a checkpoint.

Phase 5: Meta-Game & Polish (Month 5)

Goal: Implement features that encourage replayability and add a final layer of polish.

- **Week 17: Cosmetic Store (UI)**
 - [] **Store Layout:** Design and implement the UI layout for the cosmetic store. Include sections for hoverboards, character outfits, and boost trails.
 - [] **Item Display:** Create UI elements to display each cosmetic item, its cost, and its unlock status.
 - [] **UI Interactivity:** Implement button logic for selecting and purchasing items.
- **Week 18: Cosmetic Store (Backend)**

- [] **Cosmetic System:** Create a system (e.g., using ScriptableObjects) to manage all available cosmetic items and their properties.
- [] **Purchase Logic:** Link the store's UI buttons to the player's currency and data persistence systems. Deduct currency and save the unlocked state of purchased items.
- [] **Equip System:** Implement logic to allow the player to equip unlocked cosmetics, and ensure the player prefab is updated accordingly at the start of a run.
- **Week 19: Engagement Systems**
 - [] **Challenge Manager:** Create a `ChallengeManager.cs`.
 - [] **Daily/Weekly Challenges:** Implement the logic for generating simple challenges (e.g., "Collect 500 shards in one run," "Survive for 2 minutes").
 - [] **Tracking & Rewards:** Track player progress towards challenges during gameplay and award currency upon completion.
 - [] **UI for Challenges:** Create a UI element in the main menu to display active challenges.
- **Week 20: Audio Integration**
 - [] **Audio Manager:** Create an `AudioManager.cs` to handle playing all sound effects and music.
 - [] **Sound Effects (SFX):** Integrate SFX for key actions: boosting, collecting shards, collisions, UI button clicks.
 - [] **Music:** Add background music for the menu and gameplay. Implement logic to switch between tracks.
 - [] **Audio Mixing:** Adjust the volume levels of all audio sources to ensure a balanced mix.

Phase 6: Finalization & Submission (Month 6)

Goal: Crush all bugs, optimize performance, and prepare all deliverables for final submission.

- **Week 21: Alpha Testing & Feedback**
 - [] **Testing Phase:** Distribute an alpha build to a wider group of testers.
 - [] **Feedback Collection:** Systematically collect all feedback related to bugs, gameplay balance, and user experience.
 - [] **Prioritize Issues:** Create a final task list based on feedback, prioritizing game-breaking bugs first, followed by major gameplay adjustments.
- **Week 22: Performance Optimization**
 - [] **Profiling:** Use the Unity Profiler to identify performance bottlenecks (CPU and GPU).
 - [] **Draw Call Reduction:** Optimize models and materials to reduce draw calls.
 - [] **Code Optimization:** Review and refactor inefficient code, particularly in `Update()` loops and the PCG manager.
 - [] **Memory Management:** Check for memory leaks and optimize asset loading. Target a stable 60 FPS on your target devices.
- **Week 23: Bug Fixing & Final Polish**

- [] **Bug Squashing:** Work through the prioritized list of bugs and issues from the testing phase.
- [] **Polish Pass:** Go through every aspect of the game—UI, VFX, audio, controls—and add small touches to improve the overall feel and presentation.
- [] **Final Balancing:** Make final adjustments to difficulty, currency rewards, and store prices.
- **Week 24: Build, Documentation & Submission**
 - [] **Final Build:** Create the final, release-ready Android APK.
 - [] **Code Review & Cleanup:** Review all code for clarity, comments, and adherence to best practices.
 - [] **Documentation:** Prepare the final project report and presentation.
 - [] **Gameplay Video:** Record and edit a high-quality gameplay demo video.
 - [] **Submission:** Package all deliverables (Unity project, APK, GDD, report, video) and submit. Celebrate!